МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Комп’ютерний практикум № 1

з курсу «Основи розробки програмного забезпечення на платформі Microsoft.NET»

на тему: «LINQ to Objects»

Викладач:
Ліщук К.І.

Виконав:
студент групи ІП-44
Комін Іван

Київ - 2026

Комп‘ютерний практикум № 1. LINQ to Objects

Мета: - ознайомитися з обробкою даних з використанням бібліотеки LINQ to Objects

Постановка задачі комп‘ютерного практикуму № 1

При виконанні комп‘ютерного практикуму необхідно виконати наступні дії:

1) Створити програмне забезпечення, котре реалізує обробку даних з використання бібліотеки LINQ to Objects.

2) Розробити структуру даних для зберігання згідно варіантів, наведених нижче. У кожному з варіантів має бути як мінімум 4 класи. В рамках реалізації повинні бути продемонстровані зв'язки між класами: один-до-багатьох і багато-до-багатьох.

3) Розробити як мінімум 10 різних запитів, використовуючи різні дії над даними колекцій: фільтрацію за декількома критеріями, з’єднання, групування, групування разом з фільтрацією, агрегування, групування разом з агрегуванням, сортування з фільтрацією, об'єднання результатів декількох запитів в один, перетворення в інші структури, використання проміжних змінних. В запитах повинні бути використані обидва можливі варіанти реалізації LINQ-запитів (класичний варіант та з використанням методів розширення), причому запити не повинні повторюватись за діями та суттю.

4) Розробити запити згідно варіанту згідно їх словесного опису.

5) Програмне забезпечення необхідно розробити у вигляді консольного застосування на мові C#.

6) Створити звіт. Звіт повинен містити: опис архітектури проекту, словесний опис запитів, текст програмного коду, скріншоти результатів виконання.

Варіант індивідуальних завдань:

Розробити структуру даних для зберігання інформації для музичної платформи. Загальна характеристика задачі наступна:

• Користувачі слухають різні пісні.

• Пісні групуються в альбоми.

• Альбоми випускаються різними виконавцями.

• Виконавці записують багато пісень.

Розробити наступні запити:

• Визначити альбоми, у яких 80% або більше пісень мають рейтинг вище 4.

• Визначити жанри, які слухають користувачі, у яких більше 50% прослуховуваних треків тривалістю понад 5 хвилин.

• Знайти топ-3 виконавців, яких слухають користувачі, що змінили свій основний жанр музики, котру вони зазвичай слухали, за останній рік.

• Визначити альбоми, у яких більше 50% пісень потрапляють у персоналізовані плейлисти користувачів, які слухають не менше 5 жанрів.

Архітектура побудована на основі набору взаємопов'язаних класів, що моделюють доменну область музичної платформи.

Опис класів даних

1. Artist (Виконавець): Базовий клас для ідентифікації музикантів або гуртів. Містить унікальний ідентифікатор та назву.
2. Album (Альбом): Представляє музичний альбом. Містить посилання на ArtistId, назву та рік випуску. Виступає контейнером для пісень.
3. Genre (Жанр): Окремий довідник музичних напрямків. Це дозволяє уникнути дублювання назв та забезпечує цілісність даних при групуванні.
4. Song (Пісня): Центральний об'єкт даних. Містить технічні характеристики (тривалість у секундах), користувацькі характеристики (рейтинг) та зовнішні ключі до альбому та жанру.
5. User (Користувач): Зберігає інформацію про слухачів, зокрема їхні жанрові вподобання для аналізу динаміки смаків.
6. UserHistory (Історія прослуховувань): Технічний клас, що фіксує факт взаємодії користувача з конкретною піснею.

Реалізація зв'язків між даними

У проєкті продемонстровано два основні типи зв'язків, характерних для реляційних структур:

- Один-до-багатьох (One-to-Many):
    - Artist — Album: Один виконавець може мати декілька альбомів. Реалізовано через поле ArtistId у класі Album.
    - Album — Song: Один альбом містить багато треків. Реалізовано через AlbumId у класі Song.
    - Genre — Song: Кожна пісня належить до одного жанру, але один жанр може охоплювати безліч пісень.
- Багато-до-багатьох (Many-to-Many):

- User — Song: Користувачі можуть слухати багато пісень, а кожна пісня може бути у плейлистах багатьох користувачів.
- Цей зв'язок реалізовано через проміжний клас UserHistory (таблицю зв'язку), що містить пари UserId та SongId. Це дозволяє гнучко аналізувати активність користувачів без прямої залежності між класами.

Код класів:

```
public class Artist(int id, string name)
{
    public int Id { get; set; } = id;
    public string Name { get; set; } = name;

    public override string ToString() => $"Artist: {Name} (ID: {Id})";
}

public class Song(int id, int albumId, int genreId, int duration, double
rating, string title)
{
    public int Id { get; set; } = id;
    public int AlbumId { get; set; } = albumId;
    public int GenreId { get; set; } = genreId;
    public int Duration { get; set; } = duration;
    public double Rating { get; set; } = rating;
    public string Title { get; set; } = title;

    public override string ToString() => $"(ID: {Id}) {Title} -
[{Duration}s] Rating: {Rating}";
}

public class Genre(int id, string name)
{
```

```
    public int Id { get; set; } = id;
    public string Name { get; set; } = name;

    public override string ToString() => Name;
}

public class Album(int id, string title, int artistId, int year)
{
    public int Id { get; set; } = id;
    public string Title { get; set; } = title;
    public int ArtistId { get; set; } = artistId;
    public int Year { get; set; } = year;

    public override string ToString() => $"Album: '{Title}' ({Year})";
}

public class User(int id, string name, int currentGenreId)
{
    public int Id { get; set; } = id;
    public string Name { get; set; } = name;
    public int CurrentGenreId { get; set; } = currentGenreId;

    public override string ToString() => $"User: {Name} (Genre ID:
{CurrentGenreId})";
}

public class UserHistory(int userId, int songId, DateTime listenDate)
{
    public int UserId { get; set; } = userId;
    public int SongId { get; set; } = songId;
    public DateTime ListenDate { get; set; } = listenDate;
```

```
}
```

LINQ запити

1. **Фільтрація за декількома критеріями:** Запит знаходить пісні жанру «Гранж» (ID 1), тривалість яких перевищує 180 секунд (3 хвилини), а рейтинг — вище 4.5. Використовується класичний синтаксис LINQ.

```
        var grungeDurationRating =

            from song in songs

            where song.GenreId == 1 && song.Duration > 180 &&
song.Rating > 4.5

            select song;
```

```
About a Girl - 218 s - 4.8 stars
Rain When I Die - 369 s - 4.9 stars
Rooster - 311 s - 4.7 stars
```

2. **З’єднання колекцій (Inner Join):** Проводиться об'єднання списків пісень та альбомів за спільним ідентифікатором альбому. Результат — новий анонімний об'єкт, що містить назву пісні, назву її альбому та рік його випуску.

```
var songAlbumYear =

            from s in songs

            join a in albums on s.AlbumId equals a.Id

            select new

            {

                AlbumTitle = a.Title,

                SongTitle = s.Title,

                AlbumYear = a.Year,

            };
```

```
Blew from Bleach released in 1989
Floyd the Barber from Bleach released in 1989
About a Girl from Bleach released in 1989
School from Bleach released in 1989
Love Buzz from Bleach released in 1989
Rain When I Die from Dirt released in 1992
Rooster from Dirt released in 1992
Them Bones from Dirt released in 1992
Seek & Destroy from Kill 'Em All released in 1983
Hit The Lights from Kill 'Em All released in 1983
Bound To The Floor from As Good As Dead released in 1996
High Fiving MF from As Good As Dead released in 1996
The Outlaw Torn from Load released in 1996
2x4 from Load released in 1996
King Nothing from Load released in 1996
```

3. **Використання проміжних змінних :** Запит відбирає пісні тривалістю понад 5 хвилин. За допомогою ключового слова let розраховуються хвилини та секунди для створення відформатованого рядка часу.

```
var longSongsWithLet =

    from s in songs

    let mins = s.Duration / 60

    let secs = s.Duration % 60

    where mins >= 5

    select new { s.Title, Time = $"{mins}:{secs:D2}" };
```

```
Rain When I Die - 6:09
Rooster - 5:11
Seek & Destroy - 6:50
The Outlaw Torn - 9:49
2x4 - 5:30
King Nothing - 5:30
```

4. **Групування даних (Group By):** Виконується групування пісень за ідентифікатором жанру. Результат відсортовано за ключем групи (ID жанру). Це дозволяє переглядати колекції треків у розрізі кожного жанру окремо.

```
var songsByGenre = songs.GroupBy(s =>
s.GenreId).OrderBy(sGroup => sGroup.Key);
```

```
Blew - 1
Floyd the Barber - 1
About a Girl - 1
School - 1
Love Buzz - 1
Rain When I Die - 1
Rooster - 1
Them Bones - 1
King Nothing - 2
Seek & Destroy - 3
Hit The Lights - 3
Bound To The Floor - 4
High Fiving MF - 4
The Outlaw Torn - 5
2x4 - 5
```

5. **Групування з фільтрацією (Having):** Пісні групуються за назвою альбому. За допомогою агрегатної функції Average розраховується середня тривалість треків у групі, після чого відбираються лише ті альбоми, де середня тривалість перевищує 200 секунд.

```
var songsByAlbumsAvgDuration =

    from s in songs

    join a in albums on s.AlbumId equals a.Id

    group s by a.Title into g

    let avg = g.Average(s => s.Duration)

    where avg > 200

    select new { AlbumTitle = g.Key, AvgDuration = avg };
```

```
Album: Dirt - Avg Duration: 283.00s
Album: Kill 'Em All - Avg Duration: 354.50s
Album: As Good As Dead - Avg Duration: 256.00s
Album: Load - Avg Duration: 416.33s
```

6. **Агрегування значень (Sum):** Запит розраховує загальну тривалість усіх пісень у колекції. Результат використовується для виведення сумарного часу звучання медіатеки у форматі «хвилини:секунди».

```
var totalDuration = songs.Sum(s => s.Duration);
```

```
Total songs duration: 70:17
```

7. **Групування разом з агрегуванням:** Запит групує пісні за альбомами та виконує з'єднання з колекцією альбомів. Для кожної групи розраховується максимальний рейтинг пісні (Max), що дозволяє визначити найкращий трек у кожному альбомі.

```
        var maxRatingPerAlbum = songs
            .GroupBy(s => s.AlbumId)
            .Join(
                albums,
                group => group.Key,
                album => album.Id,
                (group, album) =>
                    new { AlbumTitle = album.Title, MaxRating =
group.Max(s => s.Rating) }
            );
```

```
Album: Bleach: Max rating - 4.8
Album: Dirt: Max rating - 4.9
Album: Kill 'Em All: Max rating - 4
Album: As Good As Dead: Max rating - 4.4
Album: Load: Max rating - 4.9
```

8. **Сортування з фільтрацією:** Відбираються пісні з рейтингом вище 4.0. Результати сортуються за двома критеріями: спочатку за рейтингом (у порядку спадання), а потім за назвою (за алфавітом).

```
        var sortedRatingTitle =
            from s in songs
            where s.Rating > 4
            orderby s.Rating descending, s.Title ascending
            select s;
```

```
Rain When I Die - 4.9
The Outlaw Torn - 4.9
About a Girl - 4.8
Rooster - 4.7
Blew - 4.5
Them Bones - 4.5
High Fiving MF - 4.4
Bound To The Floor - 4.2
Floyd the Barber - 4.2
School - 4.1
```

9. **Об'єднання результатів (Union):** Демонстрація операції над множинами. Об'єднуються результати двох вибірок: пісень із високим рейтингом (>4.5) та довгих пісень (>350 секунд). Метод Union автоматично видаляє дублікати, якщо пісня відповідає обом умовам.

```
var songsHighRating = songs.Where(s => s.Rating > 4.5);

var longSongs = songs.Where(s => s.Duration > 350);

var highAndLongSongs = songsHighRating.Union(longSongs);
```

```
About a Girl
Rain When I Die
Rooster
The Outlaw Torn
Seek & Destroy
```

10. **Перетворення в інші структури (ToDictionary):** Запит трансформує список об'єктів Artist у словник (Dictionary<int, string>), де ключем є ID виконавця, а значенням — його ім'я. Це оптимізує подальший пошук виконавців за ідентифікатором.

```
var artistsToDict = artists.ToDictionary(a => a.Id, a => a.Name);
```

```
ID 1: Nirvana
```

1. Визначити альбоми, у яких 80% або більше пісень мають рейтинг вище 4.

```
var goodAlbums =

    from a in albums

    join s in songs on a.Id equals s.AlbumId

    group s by a into g

    where (double)g.Count(s => s.Rating > 4) / g.Count() >= 0.8
```

```
            select new { AlbumTitle = g.Key.Title };
```

```
Bleach
Dirt
As Good As Dead
```

2. Визначити жанри, які слухають користувачі, у яких більше 50% прослуховуваних треків тривалістю понад 5 хвилин.

```
var longTrackUusers = users
            .Where(u =>
            {
                var userListenedSongs = history
                    .Where(h => h.UserId == u.Id)
                    .Join(songs, h => h.SongId, s => s.Id, (h, s) =>
s)
                    .ToList();
                if (userListenedSongs.Count() == 0)
                    return false;


                double longTracks = userListenedSongs.Count(s =>
s.Duration > 300);
                return longTracks / userListenedSongs.Count > 0.5;
            })
            .SelectMany(u => history.Where(h => h.UserId == u.Id))
            .Join(songs, h => h.SongId, s => s.Id, (h, s) =>
s.GenreId)
            .Distinct()
            .Join(genres, gId => gId, g => g.Id, (gId, g) => g.Name);
```

```
Genre: Grunge
Genre: Thrash Metal
Genre: Alternative Rock
Genre: Hard Rock
Genre: Heavy Metal
```

3. Знайти топ-3 виконавців, яких слухають користувачі, що змінили свій основний жанр музики, котру вони зазвичай слухали, за останній рік.

```
var topArtists = (
```

```
            from u in users
            // first we get the historical genre
            let historicalGenreId = history
                //get history of a user
                .Where(h => h.UserId == u.Id)
                //join with songs to get the genres
                .Join(songs, h => h.SongId, s => s.Id, (h, s) =>
s.GenreId)
                //group the genres by creating a group for each genre
id
                .GroupBy(gId => gId)
                //then order the groups by the amount of songs they
contain
                .OrderByDescending(g => g.Count())
                .Select(g => g.Key)
                //get the first one
                .FirstOrDefault()


            // then we check if the current genre is different from
the historical genre
            where historicalGenreId != 0 && u.CurrentGenreId !=
historicalGenreId


            // then we get to the artists by first getting the
history, then the song, then the album, then the artist
            join h in history on u.Id equals h.UserId
            join s in songs on h.SongId equals s.Id
            join a in albums on s.AlbumId equals a.Id
            join art in artists on a.ArtistId equals art.Id
            //group artists by name and get the most popular
            group art by art.Name into g
            orderby g.Count() descending
```

```
            select new { ArtistName = g.Key }

        ).Take(3); //then take the top 3
```

```
Artist: Alice in Chains (Streams: 5)
Artist: Nirvana (Streams: 3)
Artist: Metallica (Streams: 3)
```

4. Визначити альбоми, у яких більше 50% пісень потрапляють у персоналізовані плейлисти користувачів, які слухають не менше 5 жанрів.

```
var songsFromMultiGenreUsers = history

            .GroupBy(h => h.UserId)

            .Where(g =>

                g.Join(songs, h => h.SongId, s => s.Id, (h, s) =>
s.GenreId).Distinct().Count() >= 5

            )

            .SelectMany(g => g.Select(h => h.SongId))

            .Distinct()

            .ToList();


        var popularAlbums = albums.Where(a =>

        {

            var albumSongs = songs.Where(s => s.AlbumId ==
a.Id).ToList();

            if (albumSongs.Count == 0)

                return false;


            double favoriteCount = albumSongs.Count(s =>
songsFromMultiGenreUsers.Contains(s.Id));

            return (favoriteCount / albumSongs.Count) > 0.5;

        });
```

```
Album: Dirt
Album: Load
```

Висновок: У ході виконання лабораторної роботи було опановано технологію LINQ (Language Integrated Query) для обробки та аналізу складних структур даних у середовищі .NET. На прикладі моделі музичної платформи було реалізовано 14 запитів різного рівня складності, що охоплюють усі ключові аспекти роботи з колекціями об'єктів.